Chapter 25 | Exercise solutions with Python

Image and Video Generation

This notebook solves Exercises 5–8. It checks the endpoints of the noise mixture, follows an exact and a perturbed prediction through the same two sampler steps, and sweeps classifier-free guidance over five scales.

The arrays are small enough to compute by hand, and the code uses the chapter's local NumPy functions. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch25_visual_generation"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check the endpoints

Choose two 2×2 arrays with different entries at every position, so the two endpoint outputs are easy to tell apart. The first is the clean grid x_0. The second is the supplied noise grid ϵ.

`add_noise` multiplies the clean grid by $\sqrt{\bar{\alpha}}$ and the noise grid by $\sqrt{1-\bar{\alpha}}$, then adds the grids entry by entry. At $\bar{\alpha} = 0$ the weights are $(0, 1)$, so each entry comes from the noise grid. At $\bar{\alpha} = 1$ they are $(1, 0)$, so each entry comes from the clean grid. The same operation acts at every row and column, giving shape $(2, 2)$ in both cases.

```
from examples.ch25_visual_generation.model import add_noise

clean_grid = np.array([[2.0, -1.0], [0.5, 3.0]])
noise_grid = np.array([[-0.5, 1.5], [-1.0, 0.25]])

at_zero = add_noise(clean_grid, noise_grid, alpha_bar=0)
at_one = add_noise(clean_grid, noise_grid, alpha_bar=1)

assert at_zero.shape == (2, 2)
assert at_one.shape == (2, 2)
np.testing.assert_array_equal(at_zero, noise_grid)
np.testing.assert_array_equal(at_one, clean_grid)

print("alpha_bar = 0, shape", at_zero.shape)
print(at_zero)
print("alpha_bar = 1, shape", at_one.shape)
print(at_one)
```

```
OUTPUT
alpha_bar = 0, shape (2, 2)
[[-0.5   1.5 ]
 [-1.    0.25]]
alpha_bar = 1, shape (2, 2)
[[ 2.  -1. ]
 [ 0.5  3. ]]
```

6. Read both DDIM outputs

Start with the right-hand grid in the diffusion-path figure. Its rows are $(0.6, 0.8)$ and $(-0.8, 0.6)$. The supplied noise has rows $(0, 1)$ and $(-1, 0)$. We pass that noise to the sampler as its exact prediction.

The first output from `ddim_step` estimates the clean grid. At $\bar{\alpha}_t = 0.36$ the signal weight is 0.6 and the noise weight is 0.8. Subtracting 0.8ϵ and dividing by 0.6 recovers the clean grid with ones on the diagonal and zeros elsewhere. For example, the upper-right value becomes $(0.8 - 0.8 \cdot 1)/0.6 = 0$.

The second output is the sample at $\bar{\alpha}_s = 0.64$. To get it, multiply the clean estimate by 0.8 and add 0.6ϵ. This returns the center grid in the figure. The upper-right entry is now 0.6 because this state still contains noise. Feed this second output to the next call. At the final level, $\bar{\alpha} = 1$, the weights are 1 for signal and 0 for noise, so the final sample equals the second call's clean estimate.

```
from examples.ch25_visual_generation.model import add_noise, ddim_step

figure_clean = np.array([[1.0, 0.0], [0.0, 1.0]])
figure_noise = np.array([[0.0, 1.0], [-1.0, 0.0]])
figure_start = np.array([[0.6, 0.8], [-0.8, 0.6]])
figure_middle = np.array([[0.8, 0.6], [-0.6, 0.8]])
alpha_start, alpha_middle, alpha_final = 0.36, 0.64, 1.0

from_forward = add_noise(figure_clean, figure_noise, alpha_start)
first_clean_exact, middle_exact = ddim_step(
    figure_start, figure_noise, alpha_start, alpha_middle
)
second_clean_exact, final_exact = ddim_step(
    middle_exact, figure_noise, alpha_middle, alpha_final
)

for observed, expected in [
    (from_forward, figure_start),
    (first_clean_exact, figure_clean),
    (middle_exact, figure_middle),
    (second_clean_exact, figure_clean),
    (final_exact, figure_clean),
]:
    np.testing.assert_allclose(observed, expected, rtol=0, atol=1e-12)

print("First clean estimate")
print(first_clean_exact)
print("Sample at 0.64")
print(middle_exact)
print("Second clean estimate")
print(second_clean_exact)
print("Sample at 1.00")
print(final_exact)
```

OUTPUT

```
First clean estimate
[[1. 0.]
 [0. 1.]]
Sample at 0.64
[[ 0.8  0.6]
 [-0.6  0.8]]
Second clean estimate
[[1. 0.]
 [0. 1.]]
Sample at 1.00
[[1. 0.]
 [0. 1.]]
```

7. Change one prediction

The lower-left prediction increases from -1 to -0.9. Keep the initial grid fixed, including its lower-left value -0.8. Reuse the altered prediction on both calls. The DDIM update acts on each entry separately, so the other three entries follow their exact-prediction paths.

Let Δ mean the altered run minus the exact run at the lower-left entry. In the first call, both runs start at the same value. The first part of the DDIM equation therefore gives a clean-estimate change of $-(0.8/0.6)(0.1) = -2/15$. Subtracting the larger noise prediction makes the estimate smaller.

The remix uses the altered prediction as well. It adds $0.6(0.1)$ more noise than before, partially offsetting the change from the clean estimate. At the intermediate level, the two terms give

$$\Delta x_s = 0.8(-2/15) + 0.6(0.1) = -7/150.$$

The lower-left sample goes from -0.6 to $-97/150$.

In the second call the current sample already carries that change. The sampler subtracts the noise it just added at weight 0.6 and divides by the signal weight 0.8. The change in the second clean estimate is

$$\frac{-7/150 - 0.6(0.1)}{0.8} = -2/15.$$

The final noise weight is zero, so the final lower-left value is $-2/15$ instead of 0.

```
altered_prediction = figure_noise.copy()
altered_prediction[1, 0] += 0.1

first_clean_altered, middle_altered = ddim_step(
    figure_start, altered_prediction, alpha_start, alpha_middle
)
second_clean_altered, final_altered = ddim_step(
    middle_altered, altered_prediction, alpha_middle, alpha_final
)

clean_delta = np.array([[0.0, 0.0], [-2 / 15, 0.0]])
middle_delta = np.array([[0.0, 0.0], [-7 / 150, 0.0]])
for observed, expected in [
    (first_clean_altered - first_clean_exact, clean_delta),
    (middle_altered - middle_exact, middle_delta),
    (second_clean_altered - second_clean_exact, clean_delta),
    (final_altered - final_exact, clean_delta),
]:
    np.testing.assert_allclose(observed, expected, rtol=0, atol=1e-12)

print("Altered clean estimate at both calls")
print(np.round(first_clean_altered, 6))
print()
print("Entry          Middle exact/altered     Final exact/altered")
for (row, col), entry in zip(
    np.ndindex(figure_start.shape),
    ("Upper left", "Upper right", "Lower left", "Lower right"),
):
    print(
        f"{entry:12}  "
        f"{middle_exact[row, col]:9.6f} {middle_altered[row, col]:9.6f}"
        f"      {final_exact[row, col]:9.6f} "
        f"{final_altered[row, col]:9.6f}"
    )
```

```
OUTPUT
Altered clean estimate at both calls
[[ 1.        0.      ]
 [-0.133333  1.      ]]

Entry          Middle exact/altered     Final exact/altered
Upper left     0.800000  0.800000       1.000000  1.000000
Upper right    0.600000  0.600000       0.000000  0.000000
Lower left    -0.600000 -0.646667       0.000000 -0.133333
Lower right    0.800000  0.800000       1.000000  1.000000
```

Reusing the prediction is what preserves the first clean estimate in the second call. The second call undoes the intermediate remix with exactly the noise array used to form it. A trained denoiser would compute a fresh prediction from the intermediate sample and its noise level.

8. Sweep guidance

Use the two noise predictions from Exercise 3. At the same noisy state and step, the denoiser predicts $(0, 1)$ with the condition dropped and $(2, 0)$ with the condition present. The conditional prediction minus the unconditional prediction is $(2, -1)$. From the starting point $(0, 1)$, the scale g tells us how far to move along that vector.

The guided prediction is $(0, 1) + g(2, -1) = (2g, 1 - g)$. The five equally spaced scales are 0, 0.5, 1, 1.5, and 2. Each time g increases by 0.5, the guided vector changes by $0.5(2, -1) = (1, -0.5)$. The starting vector $(0, 1)$ cancels when we subtract one prediction from the next.

```
from examples.ch25_visual_generation.model import guided_noise

unconditional = np.array([0.0, 1.0])
conditional = np.array([2.0, 0.0])
scales = np.linspace(0, 2, num=5)
guided = np.stack([
    guided_noise(unconditional, conditional, scale)
    for scale in scales
])
differences = np.diff(guided, axis=0)

expected_guided = [
    [0, 1], [1, 0.5], [2, 0], [3, -0.5], [4, -1],
]
np.testing.assert_array_equal(guided, expected_guided)
np.testing.assert_array_equal(
    differences, np.tile([1, -0.5], (4, 1))
)

print("Scale  Guided prediction")
for scale, prediction in zip(scales, guided):
    print(f"{scale:3.1f}    {prediction}")

print("Successive differences")
print(differences)
```

OUTPUT

```
Scale  Guided prediction
0.0    [0. 1.]
0.5    [1.  0.5]
1.0    [2. 0.]
1.5    [ 3.  -0.5]
2.0    [ 4. -1.]
Successive differences
[[ 1.  -0.5]
 [ 1.  -0.5]
 [ 1.  -0.5]
 [ 1.  -0.5]]
```

In the stacked array, each row is the prediction at one scale. Taking `np.diff` along axis 0 subtracts adjacent rows, giving four identical vectors. At $g = 0$ and $g = 1$, the sweep reaches the unconditional and conditional predictions, respectively. At $g = 0.5$ it interpolates to their midpoint. At $g = 1.5$ and $g = 2$ it extrapolates beyond the conditional prediction, continuing along the same line by one and two increments, respectively.